

Constructors

The Moment of Creation

Initializing Objects with Precision

DEFAULT • PARAMETERIZED • COPY • OVERLOADING

1. What is a Constructor?

A **Constructor** is a special member function of a class that is executed automatically whenever a new object of that class is created. It acts as the "Setup Wizard" for your objects.

The Identity of a Constructor:

- **Namesake:** It must have the exact same name as the class.
- **No Return:** It does not have a return type—not even void.
- **Automatic:** You never "call" a constructor; the new or declaration keywords trigger it.

Why do we need them?

Instead of manually setting every variable after creating an object, a constructor ensures that an object is born in a "valid state" with either default or specific values.

2. The Default Constructor

This constructor takes **no arguments**. If you don't define any constructor at all, C++ provides a hidden "implicit" default constructor for you.

```
class CodeHive {
public:
    string course;

    // Default Constructor
    CodeHive() {
        course = "C++ Fundamentals";
        cout << "Setup Complete!";
    }
};

int main() {
    CodeHive myObj; // Triggers the default constructor
}
```

Observation: As soon as you write any other type of constructor, the implicit default one vanishes. You must define your own if you still want to create empty objects.

3. Parameterized Constructors

These allow you to pass values into the object at the moment it is created. This makes object creation flexible and dynamic.

```
class CodeHive {
public:
    string course;
    int duration;

    // Parameterized Constructor
    CodeHive(string c, int d) {
        course = c;
        duration = d;
    }
};

int main() {
    // Pass values directly during creation
    CodeHive pythonCourse("Python Pro", 4);
}
```

4. Cloning Objects

The **Copy Constructor** is used to create a new object as a duplicate of an existing object of the same class.

Syntax: `ClassName(const ClassName &obj)`

```
class CodeHive {
public:
    string name;
    CodeHive(string n) : name(n) {}

    // Copy Constructor
    CodeHive(const CodeHive &other) {
        name = other.name;
        cout << "Cloned " << name;
    }
};

int main() {
    CodeHive original("Logic101");
    CodeHive clone = original; // Copy constructor called
}
```

5. Dynamic Constructors

While often used for memory allocation (using `new`), a "Dynamic" constructor pattern can also refer to constructors that interact with the user or external systems during the creation phase.

```
class CodeHive {
public:
    string user;
    CodeHive() {
        cout << "Who is joining? ";
        cin >> user; // Interaction during construction
    }
};
```

This ensures that the object cannot exist without a user-provided identity.

6. Constructor Overloading

Just like function overloading, you can have multiple constructors in the same class. C++ decides which one to use based on the arguments provided.

```
class CodeHive {
public:
    CodeHive() { /* Option 1: Default */ }

    CodeHive(string c) { /* Option 2: Course provided */ }

    CodeHive(string c, int d) { /* Option 3: Both provided */ }
};

int main() {
    CodeHive c1;           // Runs Option 1
    CodeHive c2("C++");    // Runs Option 2
    CodeHive c3("C++", 12); // Runs Option 3
}
```

7. Pro Pattern: Initializer Lists

In modern C++, rather than assigning variables inside the `{ }` body, we use the **Member Initializer List**. This is faster and more professional.

```
class Player {  
    int health;  
    int ammo;  
public:  
    // This is the efficient way to initialize  
    Player(int h, int a) : health(h), ammo(a) {  
        // Body can remain empty or handle extra logic  
    }  
};
```

Why it matters: Some things (like const variables and references) CANNOT be assigned; they MUST be initialized via this list.

8. Construction on the Heap

When you create an object using `new`, the constructor is still called immediately upon memory allocation.

```
int main() {  
    // Heap allocation  
    CodeHive* ptr = new CodeHive("Advanced C++", 24);  
  
    // Cleaning up (Important!)  
    delete ptr;  
}
```

The pointer `ptr` simply stores the address, while the constructor sets up the data at that address.

9. Architectural Best Practices

1. Follow the RAII Principle

Resource Acquisition Is Initialization: Use constructors to lock resources (like files or database connections) so that if an object exists, its resources are definitely ready.

2. Keep Body Logic Minimal

Constructors should be fast. Use the Initializer List for data, and only use the `{ }` body for validation or side-effects (like logging).

3. Use 'explicit'

Mark single-parameter constructors as `explicit` to prevent the compiler from making accidental, unwanted type conversions.

10. Comprehensive Summary

Constructor Type	Usage Scenario
Default	Basic generic setup
Parameterized	Custom object creation
Copy	Duplicating objects
Overloaded	Offering multiple setup options

Skill Lab:

The Bank Task: Create a class 'Account' with a default constructor setting balance to \$0, and a parameterized constructor that allows setting a custom initial deposit. Overload them correctly.

SETUP COMPLETE: CONSTRUCTOR LOGIC MASTERED